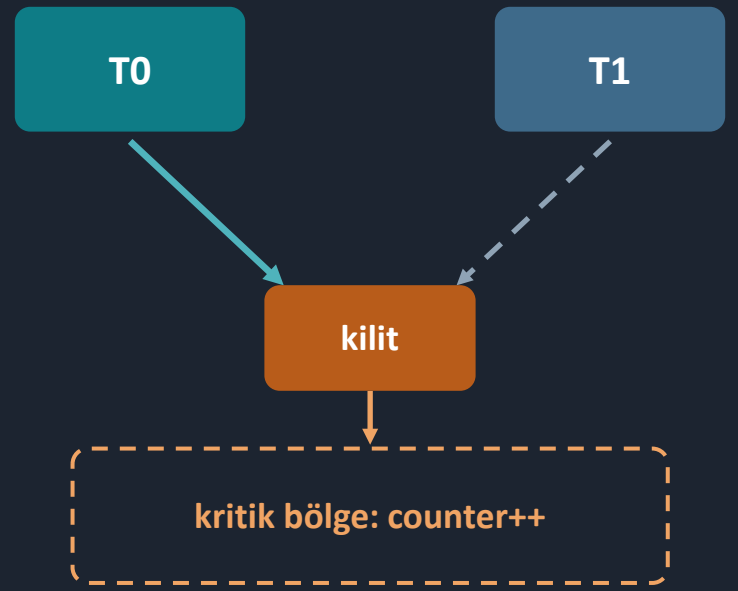
KARADENİZ TEKNİK ÜNİVERSİTESİ

Yazılım Geliştirme Bölümü

İŞLETİM SİSTEMLERİ

Hafta 6

Süreç Senkronizasyonu I: Kritik Bölge, Mutex ve Semafor



Dr. Öğr. Üyesi Sayyad Alizadeh

Bu Haftanın Yol Haritası

1

Yarış durumu

counter++ neden yanlış sonuç verebilir?

2

Kritik bölge problemi

Üç gereksinim: dışlama, ilerleme, sınırlı bekleme

3

Çekirdekte yarış

Kesintili ve kesintisiz çekirdek

4

Peterson çözümü

Yalnızca yükle/sakla ile iki süreç

5

Donanım desteği

Kesmeleri kapatma, test_and_set, CAS

6

Meşgul bekleme ve mutex

Döner kilit, acquire/release

7

Semaforlar

Sayan, ikili; sıralama; bekleme kuyruğu

8

Sorunlar

Yanlış kullanım, kilitlenme, açlık, öncelik tersinmesi

Öğrenme Çıktıları

Bu haftanın sonunda öğrenciler:

1. Bir yarış durumunun nasıl oluştuğunu iç içe geçme tablosuyla gösterir
2. Kritik bölge probleminin üç gereksinimini ayırt eder ve örneklerle açıklar
3. Peterson çözümünü adım adım izler ve modern sistemlerdeki sınırını açıklar
4. test_and_set ve compare_and_swap ile kilit mantığını açıklar
5. Meşgul bekleme, döner kilit, mutex ve semaforu karşılaştırır
6. Semafor değerini ve bekleme kuyruğunu adım adım izler; sıralama problemlerini çözer

Önceki Haftalardan Bu Haftaya

DAHA ÖNCE GÖRDÜKLERİMİZ

Paylaşımlı bellek (Hafta 3)	**İş parçacıkları ortak veri görür (Hafta 4)**
Kesintili zamanlama (Hafta 5)	**Bağlam değiştirme her an olabilir**

- Süreçler / iş parçacıkları ortak veriye erişebilir
- Bir kesme veya başka bir çekirdek olayı yürütmeyi durdurabilir; ardından CPU Zamanlayıcısı başka bir süreç/iş parçacığının çalışmasına karar verebilir.
- Birlikte düşününce: güncelleme yarıda kalırsa?

Kitapta olduğu gibi "süreç" ifadesi iş parçacıklarını da kapsar.

BU HAFTANIN TEMEL SORUSU

Bir süreç ortak veriyi güncellerken CPU elinden alınırsa ve başka bir süreç aynı veriye erişirse ne olur?

Yanıt: sonuç yürütme sırasına bağlı hâle gelebilir → yarış durumu

Motivasyon: Ortak Bir Sayaç

Üretici–tüketici süreçleri, tampondaki öğe sayısını ortak bir counter değişkeninde tutar (yalnızca motivasyon örneği).

Üretici

```
while (true) {
    /* bir öğe üret */
    while (counter == BUFFER_SIZE) ;
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

Tüketici

```
while (true) {
    while (counter == 0) ;
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
    /* öğeyi tüket */
}
```

Her iki kod ayrı ayrı doğrudur. Ama eşzamanlı çalıştıklarında counter++ ve counter-- aynı değişkeni günceller.

Silberschatz vd., Bölüm 5.1 · Tamponun doğru senkronizasyonu Hafta 7'de

counter++ Tek Bir İşlem Değildir

Kaynak kod düzeyinde tek satır; makine düzeyinde üç ayrı adım:

counter++

```
register1 = counter          /* oku     */
register1 = register1 + 1    /* değiştir */
counter   = register1        /* yaz     */
```

counter--

```
register2 = counter          /* oku     */
register2 = register2 - 1    /* değiştir */
counter   = register2        /* yaz     */
```



↑ Bu adımların arasında bir kesme (bağlam değiştirme) gerçekleşebilir

register1 ve register2 aynı fiziksel yazmaç olabilir; her süreç kendi değerini bağlam değiştirmede saklar (Hafta 3). Kaynak: Silberschatz vd., Bölüm 5.1

Adım Adım: İç İçe Geçen Yürütme

Başlangıç: counter = 5 · Üretici bir counter++, tüketici bir counter-- yapar · Doğru sonuç: 5

Adım	Üretici	Tüketici	register1	register2	counter
T0	`register1 = counter`		5	–	5
T1	`register1 = register1 + 1`		6	–	5
T2		`register2 = counter`	6	5	5
T3		`register2 = register2 – 1`	6	4	5
T4		`counter = register2`	6	4	4
T5	`counter = register1`		6	4	**6**

Sonuç: counter = 6 — beklenen 5. Tüketicinin güncellemesi kayboldu: T2'de eski değeri (5) okudu, üretici ise T5'te kendi eski hesabını yazdı.

Sarı satır: üreticinin işi yarıda kalmışken tüketicinin ortak veriyi okuduğu an

Sonuç Yürütme Sırasına Bağlıysa: Yarış Durumu

Sıralı yürütme (önce biri tamamen, sonra diğeri)	`counter = 5`	doğru
Önceki slayttaki sıra (son yazan üretici)	`counter = 6`	yanlış
T4 ile T5 yer değiştirirse (son yazan tüketici)	`counter = 4`	yanlış

Yarış durumu (race condition): Birden çok süreç aynı veriye eşzamanlı erişip değiştirdiğinde, sonucun erişimlerin hangi sırayla gerçekleştiğine bağlı olması.

Çözüm fikri: Aynı anda yalnızca bir süreç counter'ı değiştirebilmeli → süreçler senkronize edilmeli.

Kaynak: Silberschatz vd., Bölüm 5.1 (kitap örneği 4 sonucunu gösterir); Tanenbaum & Bos, Bölüm 2.3.1

Atomik İşlem (Atomic Operation)

Atomik işlem: Diğer yürütmeler açısından bölünemez, tek bir adım olarak gözlenen işlem. Ya tamamı gerçekleşmiş ya da hiç başlamamış görülür.

Atomik olmayan: counter++

Oku → değiştir → yaz adımları arasında başka bir süreç araya girebilir; ara durum (eski değer) başkalarınca görülebilir.

Atomik olsaydı

Üç adım tek bir bölünemez işlem gibi gerçekleşirdi; diğer süreç ya önceki ya da sonraki değeri görürdü, asla arasını değil.

Atomik ≠ hızlı. Atomik = başkalarının gözünde bölünemez.

Yarış durumunun temelinde oku–değiştir–yaz dizisinin atomik olmaması vardır. Kaynak: Silberschatz vd., Bölüm 5.1 ve 5.4

Kısa Bir Örnek: Ortak Banka Hesabı

Bakiye = 1000 TL. Aynı anda iki işlem: T0 +100 TL yatırır, T1 −200 TL çeker. Doğru sonuç: 900 TL.

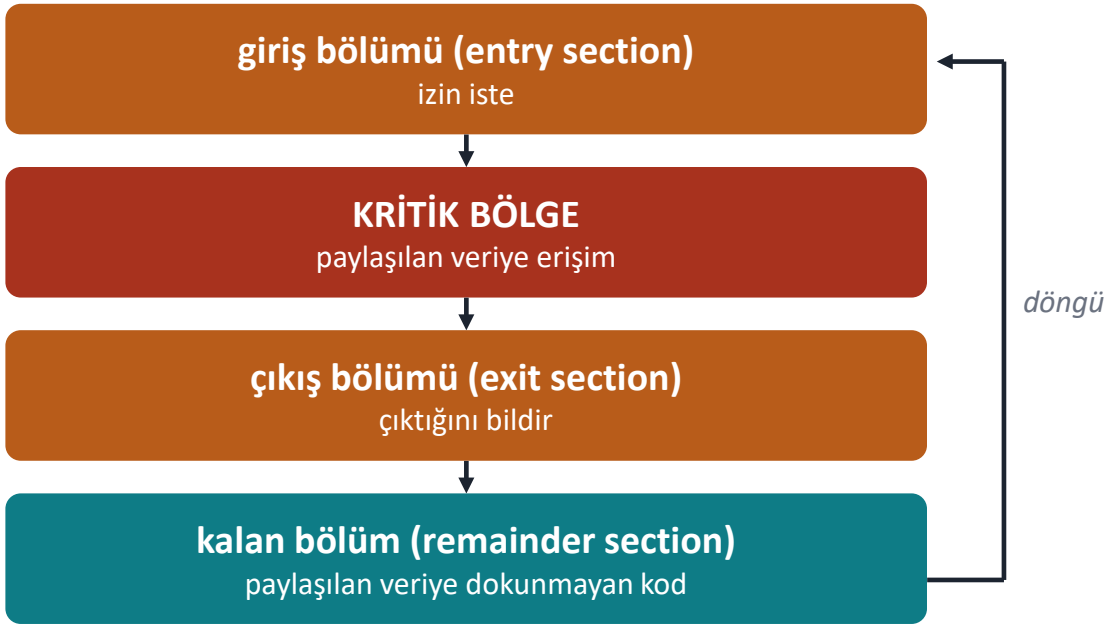
Adım	T0 (yatırma)	T1 (çekme)	bakiye
1	bakiyeyi oku: 1000		1000
2		bakiyeyi oku: 1000	1000
3	1000 + 100 = 1100 yaz		1100
4		1000 − 200 = 800 yaz	800

Yatırma kayboldu: 800 TL. Aynı yapı her ortak veride görülebilir: sayaçlar, bağlı listeler, dosya durumu, çekirdek tabloları.

Kavramsal örnek

Kritik Bölge (Critical Section)

Kritik bölge: Bir sürecin, başka süreçlerle paylaşılan veriye eriştiği ya da onu güncellediği kod bölümü. Programın tamamı değil, yalnızca bu kısımdır.




```
do {
    giriş bölümü
        KRİTİK BÖLGE
    çıkış bölümü
        kalan bölüm
} while (true);
```

Silberschatz vd., Bölüm 5.2 ve Şekil 5.1 temel alınmıştır

Kritik Bölge Problemi ve Üç Gereksinim

Problem: Süreçlerin işbirliği yapabilmesi için kritik bölgeye giriş–çıkış protokolü tasarlamak. Doğru bir çözüm üç gereksinimi sağlamalıdır.

1	**Karşılıklı dışlama** Mutual Exclusion	Bir süreç kritik bölgesindeyken başka hiçbir süreç kendi kritik bölgesinde olamaz.
2	**İlerleme** Progress	Kritik bölgede kimse yoksa ve girmek isteyenler varsa, kimin gireceği kararı sonsuza dek ertelenemez; bu karara yalnızca kalan bölümünde olmayan süreçler katılır.
3	**Sınırlı bekleme** Bounded Waiting	Bir süreç istek yaptıktan sonra, isteği karşılanana kadar diğer süreçlerin kritik bölgeye girme sayısının bir sınırı vardır.

Süreçlerin hızları hakkında hiçbir varsayım yapılmaz (yalnızca her birinin sıfırdan büyük hızla ilerlediği varsayılır). Kaynak: Silberschatz vd., Bölüm 5.2

1. Karşılıklı Dışlama (Mutual Exclusion)



Sağlar

Aynı kritik bölgeyi kullanan iki süreç asla aynı anda içeride olmaz → yarış durumu önlenir.

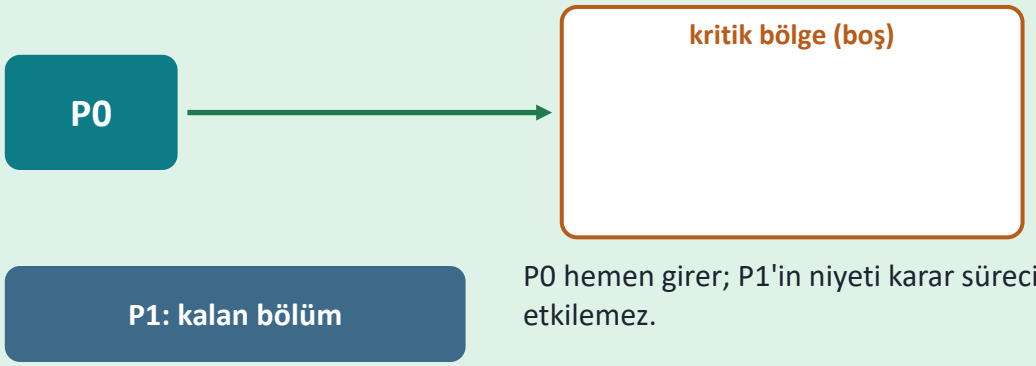
Ama tek başına yetmez

“Hiç kimse girmesin” de karşılıklı dışlamayı sağlar! İlerleme ve sınırlı bekleme de gereklidir.

2. İlerleme (Progress)

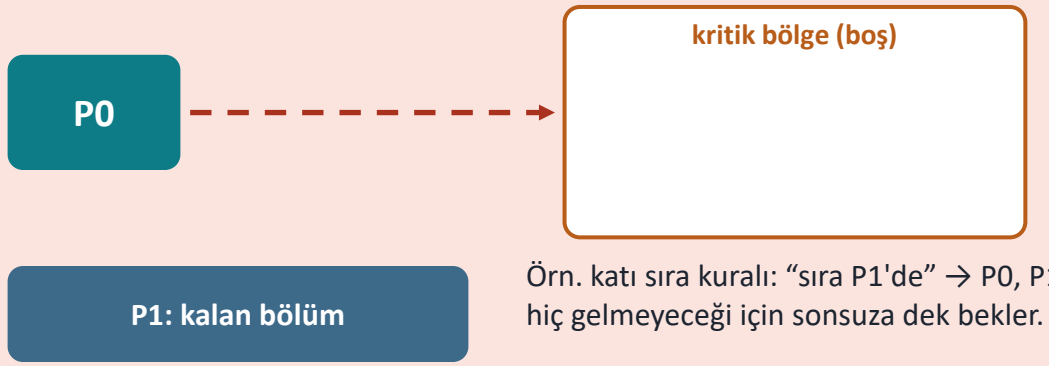
Senaryo: Kritik bölge boş. P0 girmek istiyor. P1 kalan bölümünde, girmeye hiç niyeti yok.

İlerleme sağlanıyor



P0 hemen girer; P1'in niyeti karar sürecini etkilemez.

İlerleme ihlali



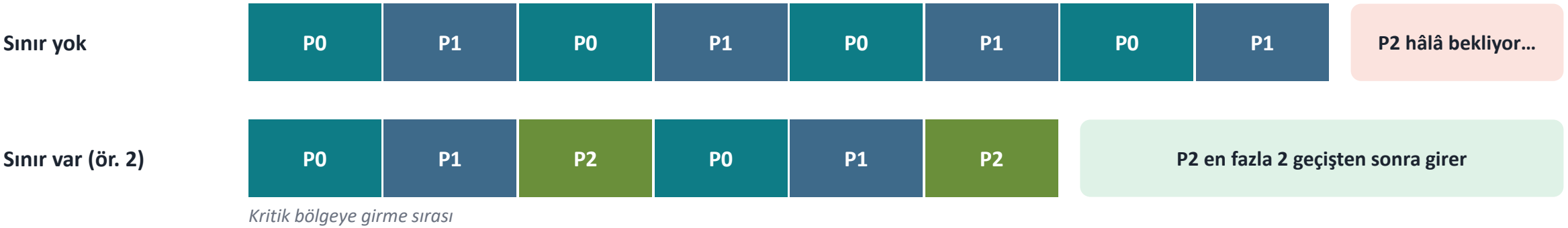
Örn. katı sıra kuralı: “sıra P1'de” → P0, P1 hiç gelmeyeceği için sonsuza dek bekler.

İlerleme: Kritik bölge boşken, girmek isteyen bir süreç gereksiz yere bekletilmemeli; karar sonsuza dek ertelenmemeli.

Katı sıra (strict alternation) örneği: Tanenbaum & Bos, Bölüm 2.3.3

3. Sınırlı Bekleme (Bounded Waiting)

Senaryo: P2 t = 0'da istek yaptı. P0 ve P1 sürekli yeniden istek yapıyor.



Tanım

İstek yapan bir süreç, başka süreçlerin kendisinden önce en fazla belirli sayıda kez kritik bölgeye girmesinden sonra mutlaka girebilmelidir.

Bağlantı: açlık (starvation)

Sınırlı bekleme ihlal edilirse bir süreç hep geçilerek sonsuza dek bekleyebilir — Hafta 5'teki açlığa benzer.

Üç Gereksinimi Karıştırmayın

Gereksinim	Neyi garanti eder?	İhlal edilirse ne olur?	Kısa sınama sorusu
Karşılıklı dışlama	Kritik bölgede aynı anda en fazla bir süreç	Yarış durumu, bozuk veri	İki süreç aynı anda içeride olabilir mi?
İlerleme	Bölge boşken isteyen biri girer; karar ertelenmez	Kimse giremez, sistem gereksiz yere durur	Bölge boşken bekleyen biri var mı?
Sınırlı bekleme	İstek yapan süreç sınırlı sayıda geçilir	Açlık: bir süreç hep geride kalır	Bir süreç sonsuza dek geçilebilir mi?

Doğru bir çözüm ÜÇÜNÜ birden sağlamalıdır. Yalnızca karşılıklı dışlama, senkronizasyon probleminin tamamı değildir.

Yarış Durumu Çekirdekte de Olur

Kitap örneği: çekirdek, sistemdeki tüm açık dosyaların listesini tutar. İki süreç aynı anda dosya açarsa iki ekleme de listeyi günceller.

Adım	P0: dosya C'yi ekler	P1: dosya D'yi ekler	liste (baş → …)
1	`C.next = baş (A)`		A → B
2		`D.next = baş (A)`	A → B
3	`baş = C`		C → A → B
4		`baş = D`	**D → A → B (C kayboldu)**

Yarışa açık diğer çekirdek veri yapıları

- **bellek ayırma yapıları**
- **süreç listeleri**
- **kesme işleme yapıları**

Liste ekleme adımları kavramsal olarak gösterilmiştir. Kaynak: Silberschatz vd., Bölüm 5.2

Kesintili ve Kesintisiz Çekirdek

Kesintisiz çekirdek

Nonpreemptive Kernel

- Çekirdek modundaki süreç; çıkana, engellenene ya da CPU'yu bırakana kadar kesilmez
- Avantajı: Tek işlemcide çekirdek veri yapılarında yarış durumunun büyük kısmı ortadan kalkar
- Dezavantajı: Uzun çalışan çekirdek kodu yanıt verebilirliği sınırlayabilir

Kesintili çekirdek

Preemptive Kernel

- Çekirdek modunda çalışan süreç de kesilebilir
- Daha iyi yanıt verebilirlik potansiyeli; gerçek zamanlı işler için daha uygun
- Ortak çekirdek verisi dikkatle korunmalıdır — özellikle çok işlemcide

Hangisinin daha iyi olduğu tasarım hedeflerine bağlıdır: kesintisiz çekirdek bazı yarışları basitleştirir; kesintili çekirdek yanıt süresini iyileştirebilir ama dikkatli kilitleme ister.

Kaynak: Silberschatz vd., Bölüm 5.2

Peterson Çözümü: İki Süreç İçin Yazılım Çözümü

Soru: Yalnızca sıradan yükle (load) / sakla (store) işlemleriyle iki süreç için karşılıklı dışlama sağlanabilir mi?

```
/* paylaşılan */
boolean flag[2];   /* başlangıçta false */
int turn;

/* Pi'nin kodu (j = 1 - i) */
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j)
        ;   /* bekle */
    /* KRİTİK BÖLGE */
    flag[i] = false;
    /* kalan bölüm */
} while (true);
```

giriş bölümü

```
flag[i] = true;
turn = j;
while(…)
```

çıkış bölümü

```
flag[i] = false;
```

İki süreç: P0 ve P1. Pi için diğer süreç Pj (j = 1 – i).

Kaynak: Silberschatz vd., Bölüm 5.3 (Şekil 5.2); Tanenbaum & Bos, Bölüm 2.3.3

flag ve turn Ne Anlama Gelir?

flag[i] = true

“Pi kritik bölgeye girmeye HAZIR / girmek İSTİYOR.” Niyeti bildirir.

turn = j

“İkimiz de istiyorsak, sıra SENDE.” Pi kibarca önceliği karşı tarafa bırakır.

Neden ikisi birlikte?

Yalnızca flag olsaydı	İki süreç de flag'ini true yapıp birbirini sonsuza dek bekleyebilirdi → ilerleme yok
Yalnızca turn olsaydı	Katı sıra: diğer süreç hiç girmek istemese de sıra gelmeden girilemez → ilerleme yok
İkisi birlikte	Yalnızca karşı taraf GERÇEKTEN istiyorsa VE sıra ondaysa beklenir

Peterson: İki Süreç Aynı Anda Girmek İsterse

Adım	Olay	flag[0]	flag[1]	turn	P0 durumu	P1 durumu
0	Başlangıç	F	F	0	kalan bölüm	kalan bölüm
1	**P0: flag[0] = true**	T	F	0	giriş	kalan bölüm
2	**P1: flag[1] = true**	T	T	0	giriş	giriş
3	**P0: turn = 1**	T	T	1	giriş	giriş
4	**P1: turn = 0**	T	T	**0**	giriş	giriş
5	**P0: flag[1] && turn==1 ? → yanlış**	T	T	0	**KRİTİK BÖLGE**	giriş
6	**P1: flag[0] && turn==0 ? → doğru**	T	T	0	**KRİTİK BÖLGE**	**bekliyor (döner)**
7	**P0: flag[0] = false**	F	T	0	kalan bölüm	bekliyor
8	**P1: flag[0] && … ? → yanlış**	F	T	0	kalan bölüm	**KRİTİK BÖLGE**

Anahtar: turn'e en SON yazan (adım 4'te P1) bekler. Böylece her ikisi aynı anda istese bile yalnızca biri girer.

F = false, T = true · Başlangıçta turn değeri önemsizdir (0 seçildi)

Peterson Üç Gereksinimi Nasıl Sağlar?

Gereksinim	Nasıl sağlanır? (sezgisel)
Karşılıklı dışlama	İkisi de içeride olsaydı flag[0] = flag[1] = true olurdu. turn aynı anda hem 0 hem 1 olamaz; turn'e son yazan süreç while döngüsünde kalır.
İlerleme	Pj girmek istemiyorsa flag[j] = false'tur ve Pi hemen girer. İkisi de istiyorsa turn değeri birini kesin olarak seçer; karar ertelenmez.
Sınırlı bekleme	Pi beklerken Pj çıkıp yeniden girmek isterse turn = i yapar; böylece Pi, Pj'nin en fazla BİR girişinden sonra girer.

Biçimsel kanıt yerine sezgisel açıklama verilmiştir. Kaynak: Silberschatz vd., Bölüm 5.3

Peterson'un Sınırı: Neden Hâlâ Öğretiliyor?

Modern sistemlerde pratik kilit DEĞİLDİR

- Modern işlemciler ve derleyiciler başarım için bağımlı olmayan okuma/yazma işlemlerini yeniden sıralayabilir
- Bu yüzden Peterson'ın modern mimarilerde doğru çalışacağının garantisi yoktur
- Yalnızca iki süreç içindir ve meşgul bekleme yapar

Neden öğreniyoruz?

- Karşılıklı dışlamanın mantığını açıkça gösterir
- Eşzamanlı yürütme hakkında akıl yürütmeyi öğretir
- Üç gereksinimi somut bir algoritmada görürüz

Pratikte çözüm: donanımın sağladığı atomik komutlar ve bunların üzerine kurulan kilitler → sonraki bölüm.

Donanım Desteği I: Kesmeleri Kapatmak

Soru: İki komut arasına başka süreç giriyorsa, donanım bize yardım edebilir mi?

kesmeleri kapat → **kritik bölge** → **kesmeleri aç**

Tek işlemcide: kesme yoksa Donanım Zamanlayıcısı kesmesi de gelmez → CPU başka sürece geçemez → komut dizisi kesintisiz çalışır.

CPU0: kesmeler kapalı · counter++

CPU1: çalışmaya devam · counter--

çok işlemcide işe yaramaz

Neden genel bir çözüm değil?

Kullanıcı programına bu yetki verilemez (ayrıcalıklı komut, Hafta 1)

Uzun süre kapalı kalırsa sistem olaylara (ör. saat, G/Ç) yanıt veremez

Çok işlemcide diğer CPU'lar çalışmaya devam eder; tüm CPU'lara mesaj göndermek pahalıdır

Yalnızca çekirdek içinde, kısa bölümler için kullanılabilen bir tekniktir. Kaynak: Silberschatz vd., Bölüm 5.4; Tanenbaum & Bos, Bölüm 2.3.3

Atomik Komut: test_and_set()

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;    /* eski değer */
    *target = true;          /* her zaman true yap */
    return rv;
}
```

Tüm gövde donanım tarafından ATOMİK olarak yürütülür

İki CPU aynı anda çağırsa bile işlemler sırayla gerçekleşir.

Kilit olarak kullanımı (lock başlangıçta false)

```
do {
    while (test_and_set(&lock))
        ;                    /* döner: lock alınamadı */
    /* KRİTİK BÖLGE */
    lock = false;            /* kilidi bırak */
    /* kalan bölüm */
} while (true);
```

`dönüş false:` kilit boştu ve şimdi bende → gir

`dönüş true:` kilit başkasında → tekrar dene

Kavramsal sözde kod; gerçek işlemci komutları farklı adlar taşır. Kaynak: Silberschatz vd., Bölüm 5.4 (Şekil 5.3–5.4)

test_and_set İzleme: İki İş Parçacığı Aynı Kilidi İster

Adım	Olay	dönüş	lock (sonra)	T0	T1
0	Başlangıç	–	false	kalan bölüm	kalan bölüm
1	**T0: test_and_set(&lock)**	false	true	**KRİTİK BÖLGE**	kalan bölüm
2	**T1: test_and_set(&lock)**	true	true	kritik bölge	**döner**
3	**T1: test_and_set(&lock)**	true	true	kritik bölge	**döner**
4	**T0: lock = false**	–	false	kalan bölüm	döner
5	**T1: test_and_set(&lock)**	false	true	kalan bölüm	**KRİTİK BÖLGE**
6	**T1: lock = false**	–	false	kalan bölüm	kalan bölüm

Atomiklik sayesinde adım 1 ile 2 arasında ikisi birden false göremez: en fazla bir iş parçacığı kilidi alır.

T1 beklerken CPU kullanmaya devam eder (meşgul bekleme).

Atomik Komut: compare_and_swap() (CAS)

```
int compare_and_swap(int *value, int expected,
                     int new_value) {
    int temp = *value;          /* eski değer */
    if (*value == expected)
        *value = new_value;     /* eşitse değiştir */
    return temp;                /* eski değeri döndür */
}
```

Tüm işlem ATOMİKTİR

"Değer hâlâ beklediğim gibiyse güncelle; değilse dokunma."

Kilit olarak kullanımı (lock = 0 boş, 1 dolu)

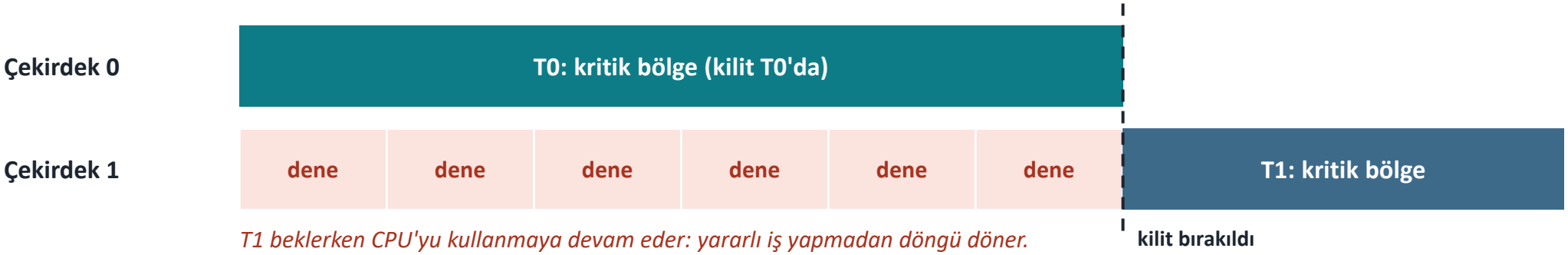
```
while (compare_and_swap(&lock, 0, 1) != 0)
    ;                           /* döner */
/* KRİTİK BÖLGE */
lock = 0;
```

Olay	lock	dönüş	sonuç
T0: CAS(&lock, 0, 1)	0 → 1	0	girer
T1: CAS(&lock, 0, 1)	1 → 1	1	döner
T0: lock = 0	1 → 0	–	çıkar
T1: CAS(&lock, 0, 1)	0 → 1	0	girer

Kaynak: Silberschatz vd., Bölüm 5.4 (Şekil 5.5–5.6)

Meşgul Bekleme (Busy Waiting)

Kilit boşalana kadar döngü içinde sürekli denetlemek: while (lock) ;



T1 beklerken CPU'yu kullanmaya devam eder: yararlı iş yapmadan döngü döner.

Olası avantaj

Bekleme çok kısaysa bağlam değiştirme maliyeti oluşmaz.

Dezavantaj

Bekleme uzunsa CPU zamanı boşa harcanır; tek işlemcide kilidi tutan süreç çalışamadığı için daha da kötüdür.

Not: Basit test_and_set / CAS kilitleri sınırlı beklemeyi garanti etmez; kitap, waiting[] dizisiyle sınırlı beklemeyi sağlayan bir sürüm verir (Bölüm 5.4).

Döner Kilit (Spinlock)

Döner kilit: Meşgul bekleme kullanan kilit; bekleyen iş parçacığı kilidin boşalmasını “dönerek” bekler.

Yararlı olabildiği durum

- Çok işlemcili sistem: bir iş parçacığı bir çekirdekte dönerken diğeri başka çekirdekte kritik bölgeyi bitirir
- Kilit çok kısa süre tutuluyorsa
- Bekleme süresi iki bağlam değiştirmeden kısaysa

Uygun olmadığı durum

- Tek işlemcili sistem: kilidi tutan çalışamazken dönmek boşunadır
- Kilit uzun süre tutuluyorsa
- Çok sayıda iş parçacığı aynı kilit için dönüyorsa

“Döner kilit her zaman iyidir / kötüdür” denemez: bekleme süresine, işlemci sayısına ve kritik bölgenin uzunluğuna bağlıdır.

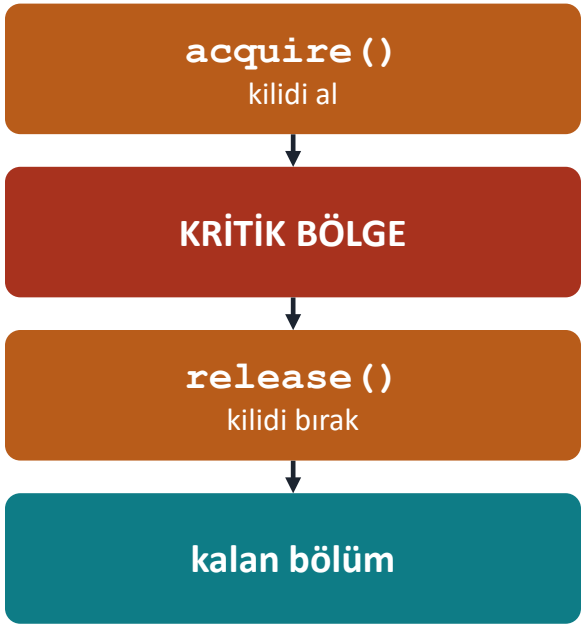
Kaynaklar: Silberschatz vd., Bölüm 5.5; Tanenbaum & Bos, Bölüm 8.1.3 (çok işlemcide senkronizasyon)

Mutex Kilidi: acquire() ve release()

Donanım komutları uygulama programcısı için karmaşıktır; İS daha üst düzeyde bir araç sunar: mutex (mutual exclusion) kilidi.

```
acquire() {
    while (!available)
        ;          /* meşgul bekleme */
    available = false;
}

release() {
    available = true;
}
```



- available: kilit boş mu?
- acquire ve release ATOMİK olmalıdır — genellikle test_and_set / CAS ile gerçekleştirilir
- Bu sürüm meşgul bekleme yapar: bir döner kilittir

Kaynak: Silberschatz vd., Bölüm 5.5 · Engelleyen (bekleten) mutex gerçekleştirmeleri de vardır; bekleme kuyruğu fikri semaforlarda

Mutex: Doğru ve Yanlış Kullanım

Doğru

```
acquire(&kilit);
counter++;
release(&kilit);
```

Ortak verinin tüm erişimi kilit içinde

Yanlış: kilit dışında erişim

```
counter++;
acquire(&kilit);
/* … */
release(&kilit);
```

counter++ korunmuyor → yarış durumu sürer

Yanlış: release unutuldu

```
acquire(&kilit);
counter++;
/* release yok */
```

Kilit hiç bırakılmaz → diğerleri sonsuza dek bekler

Kural: Aynı ortak veriye erişen TÜM kod yolları AYNI kilidi kullanmalıdır. Bir yolun bile kilitsiz erişmesi korumayı bozar.

Kilit Ne Kadar Geniş Olmalı? (Kısa Bakış)

Çok büyük kritik bölge

```
acquire(&k);
dosyayı_oku();      /* uzun */
counter++;
release(&k);
```

- Kod basit
- Gereksiz yere uzun tutulur; eşzamanlılık azalır

Yalnızca gereken kısım

```
dosyayı_oku();
acquire(&k);
counter++;
release(&k);
```

- Kilit kısa tutulur; diğerleri daha az bekler
- Çok parçalı kilitleme ise kodu karmaşıklaştırır, daha fazla kilit işlemi gerekir

İlke: kilit, ortak verinin tutarlılığı için gereken en küçük bölümü kapsamalıdır — ne eksik ne fazla.

Semafor (Semaphore) Nedir?

Semafor S: Başlangıç değeri verildikten sonra yalnızca iki atomik işlemle erişilen bir tamsayı senkronizasyon değişkeni.

`wait(S)`	`signal(S)`
Dijkstra: P (proberen, "dene") · diğer adı: down	*Dijkstra: V (verhogen, "artır") · diğer adı: up*
Değer uygun değilse bekle; uygunsa azalt	**Değeri artır; bekleyen varsa birini uyandır**

Bu sunumda ana terimler: wait() ve signal(). Semafor, kilitlerden daha genel bir araçtır: hem karşılıklı dışlama hem de sıralama için kullanılır.

Kaynaklar: Silberschatz vd., Bölüm 5.6; Tanenbaum & Bos, Bölüm 2.3.5

Klasik Tanım (Meşgul Beklemeli)

```
wait(S) {
    while (S <= 0)
        ;   /* dön */
    S--;
}

signal(S) {
    S++;
}
```

ÇOK ÖNEMLİ

wait() ve signal() içindeki tüm değişiklikler BÖLÜNEMEZ biçimde yürütülmelidir. Bu sözde kod sıradan, bölünebilir C kodu gibi okunmamalıdır.

- Bir süreç S'yi değiştirirken başka biri aynı anda değiştiremez
- wait() içinde S ≤ 0 testi ve S-- arasına kimse giremez

Bu klasik tanımda S hiçbir zaman negatif olmaz. Daha sonra göreceğimiz bekleme kuyruklu gerçekleştirmede S negatif olabilir.

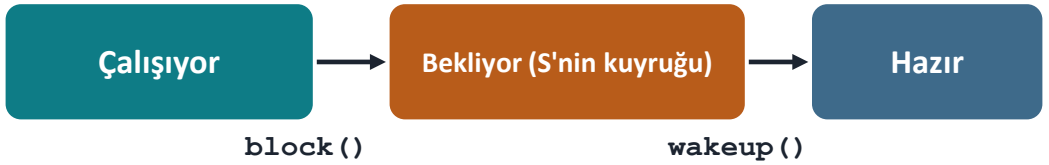
Bekleme Kuyruklu Semafor: Bu Sunumdaki Kural

```
wait(S) {
    S->value--;
    if (S->value < 0) {
        bu süreci S->list'e ekle;
        block();            /* Bekliyor durumu */
    }
}
signal(S) {
    S->value++;
    if (S->value <= 0) {
        S->list'ten bir P çıkar;
        wakeup(P);          /* Hazır durumu */
    }
}
```

Tüm izlerde kullanılan TEK kural (kitap)

- Önce azalt, sonra sına
- value < 0 ise: |value| = bekleyen süreç sayısı
- Bekleme kuyruğu FIFO kabul edilir

Süreç durumlarıyla bağlantı (Hafta 3)



Meşgul bekleme yerine engelleme: bekleyen süreç CPU kullanmaz. Kaynak: Silberschatz vd., Bölüm 5.6.2

Sayan Semafor (Counting Semaphore): 3 Yazıcı

S = 3 (kullanılabilir yazıcı sayısı) · Yazıcı almak: wait(S) · bırakmak: signal(S)

Başlangıç	S = 3	boş	boş	boş		kuyruk: boş
P1 wait	S = 2	dolu	boş	boş	P1 yazıcı alır	kuyruk: boş
P2 wait	S = 1	dolu	dolu	boş	P2 yazıcı alır	kuyruk: boş
P3 wait	S = 0	dolu	dolu	dolu	P3 yazıcı alır	kuyruk: boş
P4 wait	S = −1	dolu	dolu	dolu	yazıcı yok → P4 bekler	kuyruk: P4

Sayan semaforun değeri herhangi bir tamsayı olabilir; aynı türden birden çok kaynağı yönetir. Bir yazıcı bırakan signal(S) yaparsa S = 0 olur ve P4 uyandırılır.

İkili Semafor (Binary Semaphore) ve Mutex

İkili semafor

Değeri yalnızca 0 ya da 1 olan semafor (klasik tanımda). 1 ile başlatılınca karşılıklı dışlama için kullanılabilir.

Mutex kilidi

Yalnızca karşılıklı dışlama için tasarlanmış kilit: al (acquire) – bırak (release).

Ölçüt	İkili semafor	Mutex kilidi
Benzerlik	İkisi de aynı anda tek sürecin kritik bölgeye girmesini sağlayabilir	
Kullanım amacı	Karşılıklı dışlama ve sıralama (sinyalleşme)	Yalnızca karşılıklı dışlama
Kim serbest bırakır?	signal() farklı bir süreç tarafından da çağrılabilir	Kavramsal olarak kilidi alan süreç bırakır

Sahiplik kuralları sistemden sisteme değişebilir; burada kavramsal fark verilmiştir. Kaynak: Silberschatz vd., Bölüm 5.6.1

Semaforla Karşılıklı Dışlama: Zaman Çizelgesi

```
semaphore mutex = 1;

wait(mutex);
    /* KRİTİK BÖLGE */
signal(mutex);
```



Adım	İşlem	mutex	Kritik bölgede	Bekleme kuyruğu
0	Başlangıç	1	–	boş
1	**P0: wait(mutex)**	0	P0	boş
2	**P1: wait(mutex)**	−1	P0	P1
3	**P0: signal(mutex) → P1 uyanır**	0	P1	boş
4	**P1: signal(mutex)**	1	–	boş

Kesikli çizgiler adım 1–4'ü gösterir. Herhangi bir anda kritik bölgede en fazla bir süreç vardır.

Semafor Yalnızca Dışlama İçin Değildir: Sıralama

Problem: P1'deki S1 ifadesi, P2'deki S2 ifadesinden MUTLAKA önce çalışmalı.

```
semaphore sync = 0;
```

P1

```
S1;
signal(sync);
```

P2

```
wait(sync);
S2;
```

Neden 0? “Henüz S1 olmadı” durumunu temsil eder: P2 başlangıçta geçemez; ancak P1'in signal()'ı bir “izin” üretir.

sync = 1 olsaydı P2, S1'i beklemeden S2'yi çalıştırabilirdi → sıralama bozulur.

Çözümlü Örnek: Önce Yükle, Sonra İşle

```
semaphore ready = 0;

/* İş parçacığı A */        /* İş parçacığı B */
dosyayı_yükle();            wait(ready);
signal(ready);              dosyayı_işle();
```

Başlangıç 0: “dosya henüz hazır değil”

Durum 1: B önce çalışır

Olay	ready	B
B: wait(ready)	−1	bekliyor
A: dosyayı_yükle()	−1	bekliyor
A: signal(ready)	0	uyanır
B: dosyayı_işle()	0	işler

Durum 2: A önce çalışır

Olay	ready	B
A: dosyayı_yükle()	0	–
A: signal(ready)	1	–
B: wait(ready)	0	geçer
B: dosyayı_işle()	0	işler

Her iki sırada da dosyayı_işle(), dosyayı_yükle()'den sonra gerçekleşir.

Semafor İzleme: Başlangıç S = 2

Dört süreç aynı kaynağı (2 kopya) kullanır. Kural: önce azalt; S < 0 ise bekle · signal'da S ≤ 0 ise kuyruğun başındakini uyandır (FIFO).

Adım	İşlem	S	Kaynağı kullanan	Bekleme kuyruğu	Açıklama
0	Başlangıç	2	–	boş	iki kopya boş
1	**P1: wait(S)**	1	P1	boş	S ≥ 0 → devam
2	**P2: wait(S)**	0	P1, P2	boş	S ≥ 0 → devam
3	**P3: wait(S)**	−1	P1, P2	P3	S < 0 → P3 bekler
4	**P4: wait(S)**	−2	P1, P2	P3, P4	2 süreç bekliyor
5	**P1: signal(S)**	−1	P2, P3	P4	S ≤ 0 → P3 uyanır
6	**P2: signal(S)**	0	P3, P4	boş	S ≤ 0 → P4 uyanır
7	**P3: signal(S)**	1	P4	boş	bekleyen yok
8	**P4: signal(S)**	2	–	boş	başlangıç durumu

Denetim: her adımda S < 0 ise bekleyen sayısı = |S|; S ≥ 0 ise kaynağı kullanan sayısı = 2 − S.

wait() İşleminin Kendisinde Yarış Olursa?

S = 1 iken iki süreç aynı anda wait(S) çağırsın ve S-- atomik olmasın:

Adım	P0	P1	S
1	S'yi oku: 1		1
2		S'yi oku: 1	1
3	0 yaz, S ≥ 0 → gir		0
4		0 yaz, S ≥ 0 → gir	**0**

İkisi de kritik bölgeye girer: semafor kendi işini yapamaz! (Doğrusu: S = −1 ve P1 beklemeli.)

Tek işlemcide

wait/signal yürütülürken kesmeler kısa süre kapatılabilir.

Çok işlemcide

compare_and_swap ya da döner kilitlerle korunur — meşgul bekleme yalnızca bu çok kısa bölüme taşınır.

Mutex, Döner Kilit ve Semafor Karşılaştırması

Araç	Temel amaç	Bekleme biçimi	Tipik kullanım	Avantaj	Risk / maliyet
Mutex kilidi	Karşılıklı dışlama	Gerçekleştirmeye göre dönme ya da engellenme	Tek bir ortak veriyi korumak	Basit, anlaşılır arayüz	release unutulursa diğerleri bekler
Döner kilit	Karşılıklı dışlama	Meşgul bekleme	Çok işlemcide kısa kritik bölgeler	Bağlam değiştirme yok	Uzun beklemede CPU israfı
Sayan semafor	N kopyalı kaynağı yönetmek	Engellenme (kuyruklu gerçekleştirme)	Havuzlar, sınırlı kaynaklar	Sayma ve sıralama yeteneği	Başlangıç değeri ve sıra hataları
İkili semafor	Dışlama ya da sinyalleşme	Engellenme (kuyruklu gerçekleştirme)	Kritik bölge, olay bildirme	Hem dışlama hem sıralama	Yanlış wait/signal sırası

Mutlak bir “en iyi” yoktur: seçim; bekleme süresine, işlemci sayısına ve problemin türüne (dışlama mı, sıralama mı, sayma mı) bağlıdır.

Bu Kodlarda Problem Nedir?

Kod 1

```
signal(mutex);
  /* KRİTİK BÖLGE */
wait(mutex);
```

Sıra ters: birden çok süreç aynı anda kritik bölgeye girebilir → karşılıklı dışlama bozulur

Kod 2

```
wait(mutex);
  /* KRİTİK BÖLGE */
wait(mutex);
```

signal yerine wait: süreç kendini kalıcı olarak bekletir; sonraki herkes de bekler

Kod 3

```
semaphore sync = 1;
/* P2 */ wait(sync); S2;
/* P1 */ S1; signal(sync);
```

Başlangıç değeri yanlış: P2, S1'i beklemeden S2'yi çalıştırabilir → sıralama bozulur

Bir başka hata: wait() ya da signal()'ın unutulması. Bu hatalar zamanlamaya bağlı olduğundan her çalıştırmada görülmeyebilir — bulmak zordur.

Monitörler bu tür hataları azaltmak için geliştirilmiştir → Hafta 7

Kilitlenmeye (Deadlock) Ön Bakış

```
/* S = 1, Q = 1 */

/* P0 */          /* P1 */
wait(S);          wait(Q);
wait(Q);          wait(S);
  …                 …
signal(S);        signal(Q);
signal(Q);        signal(S);
```

Adım	Olay	S	Q	Durum
1	P0: wait(S)	0	1	P0 devam
2	P1: wait(Q)	0	0	P1 devam
3	P0: wait(Q)	0	−1	P0 bekler (Q'yu P1 tutuyor)
4	P1: wait(S)	−1	−1	P1 bekler (S'yi P0 tutuyor)

P0 ile P1 birbirini sonsuza dek bekler: Kilitlenme (Deadlock) oluşabilir. Signal yapabilecek tek süreç, kendisi beklemektedir.

Bu yalnızca bir ön bakıştır. Kilitlenmenin koşulları ve çözümleri: Hafta 8.

Kaynak: Silberschatz vd., Bölüm 5.6.3

Açlık (Starvation) Kilitlenme Değildir

Semaforun bekleme kuyruğu adil değilse (ör. LIFO: son gelen ilk uyandırılır):



Ölçüt	Kilitlenme (Deadlock)	Açlık (Starvation)
Ne olur?	Bir küme süreç birbirini sonsuza dek bekler	Bir süreç sürekli geçilerek sonsuza dek bekleyebilir
Sistem ilerler mi?	İlgili süreçler hiç ilerlemez	Diğer süreçler ilerler; mağdur süreç ilerlemez
Tipik neden	Döngüsel kaynak bekleme	Adil olmayan seçim politikası

FIFO bekleme kuyruğu bu tür açlığı önler. Kaynak: Silberschatz vd., Bölüm 5.6.3

Öncelik Tersinmesi (Priority Inversion)

L kilidi tutarken H kilidi bekler; kilitle ilgisi olmayan M, L'yi keser



Ne oldu?

H, M'den yüksek öncelikli olduğu hâlde M'nin bitmesini dolaylı olarak bekler (t = 4–9). Bekleme süresi L'ye değil M'ye bağlı hâle gelir.

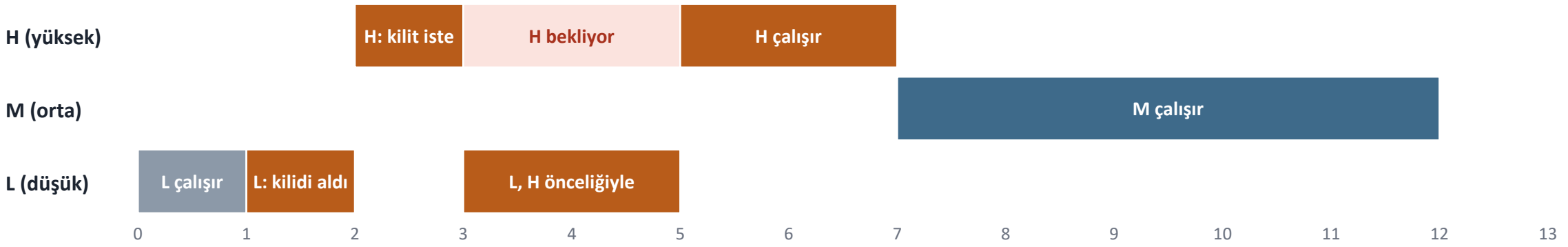
Koşul

Üçten fazla öncelik düzeyi olan ve paylaşılan kaynakların kilitle korunduğu sistemlerde ortaya çıkabilir.

Kavramsal zaman çizelgesi (birim zaman). Kaynak: Silberschatz vd., Bölüm 5.6.4

Çözüm: Öncelik Kalıtımı (Priority Inheritance)

L, H'nin beklediği kilidi tutarken geçici olarak H'nin önceliğini devralır



t = 4'te M gelir ama L'yi kesemez (L şimdi yüksek öncelikli)

Mekanizma

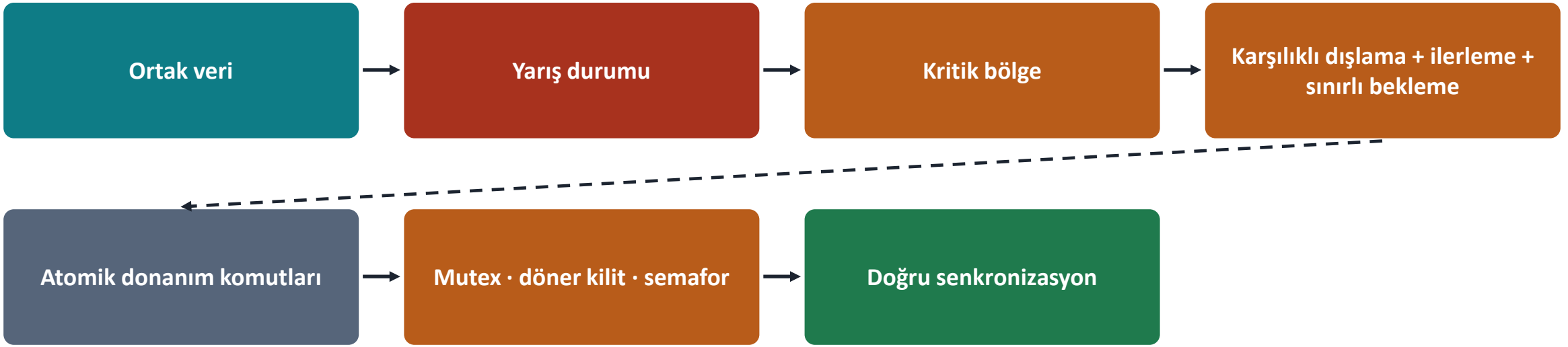
- L kilidi tutarken H'nin önceliğini devralır
- L kritik bölgeyi bitirip kilidi bırakır (t = 5)
- L asıl önceliğine döner; H hemen çalışır

Karşılaştırma

H'nin bitişi: kalıtım yokken t = 12, kalıtımla t = 7. M, H'den sonra çalışır.

Kitapta Mars Pathfinder örneği anlatılır. Kaynak: Silberschatz vd., Bölüm 5.6.4

Haftanın Kavram Haritası



Peterson: mantığı öğretir; modern sistemde garanti yok

Meşgul bekleme ↔ engellenme: kısa bekleme ↔ uzun bekleme

Riskler: kilitlenme, açlık, öncelik tersinmesi

Gelecek hafta: Bu araçlarla klasik problemler (sınırlı tampon, okuyucu–yazar, filozoflar) ve monitörler

Yaygın Yanlış Anlamalar

YANLIŞ	DOĞRU
“counter++ tek bir atomik işlemdir”	Oku–değiştir–yaz: üç adım; aralarında kesilebilir
“Karşılıklı dışlama her şeyi çözer”	İlerleme ve sınırlı bekleme de sağlanmalıdır
“Kritik bölge tüm programdır”	Yalnızca ortak veriye erişen kod bölümüdür
“Peterson pratik bir modern kilittir”	Modern mimarilerde doğru çalışma garantisi yoktur
“Meşgul bekleyen süreç CPU kullanmaz”	Döngü dönerek CPU zamanı harcar
“Döner kilit her zaman iyi / kötüdür”	Bekleme süresine ve işlemci sayısına bağlıdır
“Semafor yalnızca dışlama içindir”	Sıralama ve kaynak sayma için de kullanılır
“Sayan semafor yalnızca 0 ve 1 alır”	Herhangi bir tamsayı olabilir; 0/1 olan ikili semafordur
“wait/signal atomik olmak zorunda değil”	Atomik değilse semaforun kendisinde yarış oluşur
“Kilitlenme = açlık”	Kilitlenmede bir küme hiç ilerlemez; açlıkta mağdur süreç geçilir

Kontrol Soruları

İÇ İÇE GEÇME · GEREKSİNİMLER

S1

Yandaki sırayla yürütme sonunda counter kaç olur? Doğru sonuç kaç olmalıydı? Hangi güncelleme kaybolur?

Her adımda r1, r2 ve counter değerlerini tablo hâlinde yazın.

S2

“Kimse kritik bölgeye giremiyor ama asla iki süreç aynı anda içeride değil.” Hangi gereksinim sağlanıyor, hangisi ihlal ediliyor?

İlgili bölüm: 2 · Kritik bölge problemi

İç içe geçme (counter = 10)

```
T0: r1 = counter
T1: r2 = counter
T1: r2 = r2 + 1
T1: counter = r2
T0: r1 = r1 + 1
T0: counter = r1
```

T0 ve T1 ikisi de counter++ yapar

Kontrol Soruları

PETERSON · ATOMİK KOMUTLAR

S3 **Peterson çözümünde P1 önce flag[1] = true, turn = 0 yapsın; ardından P0 flag[0] = true, turn = 1 yapsın. Hangi süreç kritik bölgeye girer? Değişken tablosu ile gösteriniz.**

Anahtar: turn'e en son kim yazdı?

S4 **lock = false iken T0 ve T1 sırasıyla test_and_set(&lock) çağırır. Her çağrının dönüş değeri ve sonraki lock değeri nedir?**

İlgili bölüm: 4 · Donanım desteği

Kontrol Soruları

MUTEX · DÖNER KİLİT

S5 **Tek işlemcili bir sistemde döner kilit kullanmak neden çoğu zaman verimsizdir? Çok işlemcide hangi koşulda yararlı olabilir?**

Kilidi tutan süreç bekleyen dönerken çalışabilir mi?

S6 **Bir programcı counter++ işlemini korumak için acquire()'ı counter++'tan sonra çağırıyor. Ne olur?**

İlgili bölüm: 5 · Mutex

Kontrol Soruları

SEMAFOR İZLEME · SIRALAMA

S7 **S = 2 iken sırasıyla: P1 wait, P2 wait, P3 wait, P1 signal, P4 wait, P2 signal. Her adımda S değerini, kaynağı kullananları ve bekleme kuyruğunu yazınız.**

Bu sunumdaki kural: önce azalt; S < 0 ise bekle · signal'da S ≤ 0 ise kuyruk başını uyandır.

S8 **“P1'deki S1, P2'deki S2'den önce gerçekleşmeli.” Hangi semaforu, hangi başlangıç değeriyle kullanırsınız? wait() ve signal() nereye yerleştirilmeli?**

Başlangıç değerinin anlamını düşünün.

Kontrol Soruları

ÖNCELİK TERSİNMESİ · KAVRAMSAL

S9 **L kilidi tutarken H kilidi bekliyor, M ise L'yi kesiyor. Öncelik kalıtımı olmadan ve olarak H'nin ne zaman çalışabileceğini kısa bir zaman çizelgesiyle açıklayınız.**

İlgili bölüm: 7 · Öncelik tersinmesi

S10 **Kilitlenme ile açlık arasındaki farkı semafor örnekleriyle açıklayınız.**

İlgili bölüm: 7 · Sorunlar

Kaynaklar

ANA KAYNAK

Silberschatz, A., Galvin, P. B., & Gagne, G. (2013). *Operating System Concepts* (9th ed.). John Wiley & Sons.

Bu hafta: Bölüm 5 — 5.1–5.6

DESTEKLEYİCİ KAYNAK

Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson.

Bu hafta: Bölüm 2 — 2.3.1–2.3.6 · Bölüm 8.1.3 (çok işlemcide senkronizasyon)

Not: Yürütme izleri ve zaman çizelgeleri kavramsaldır; kitaptaki algoritmalar temel alınarak hazırlanmıştır.